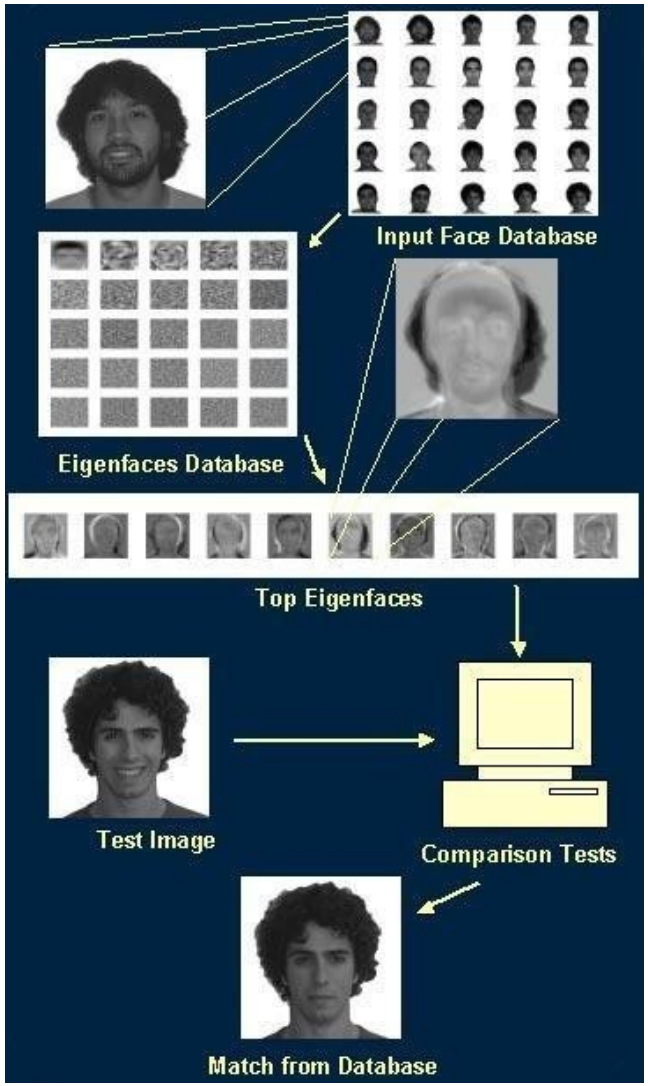
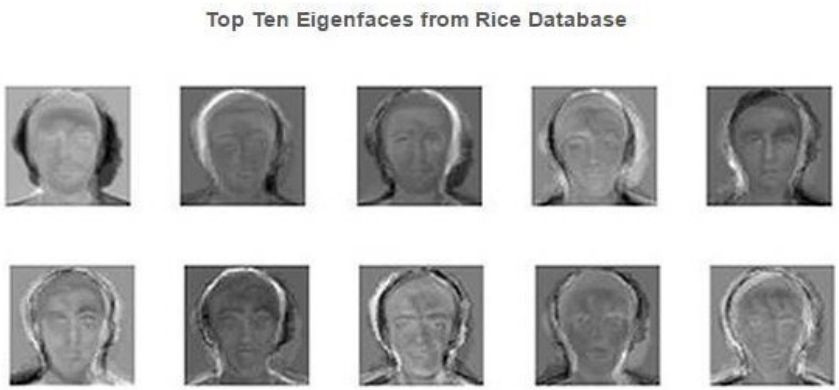
Tugas I EB3203 - Improving Eigenface Recognition Rate

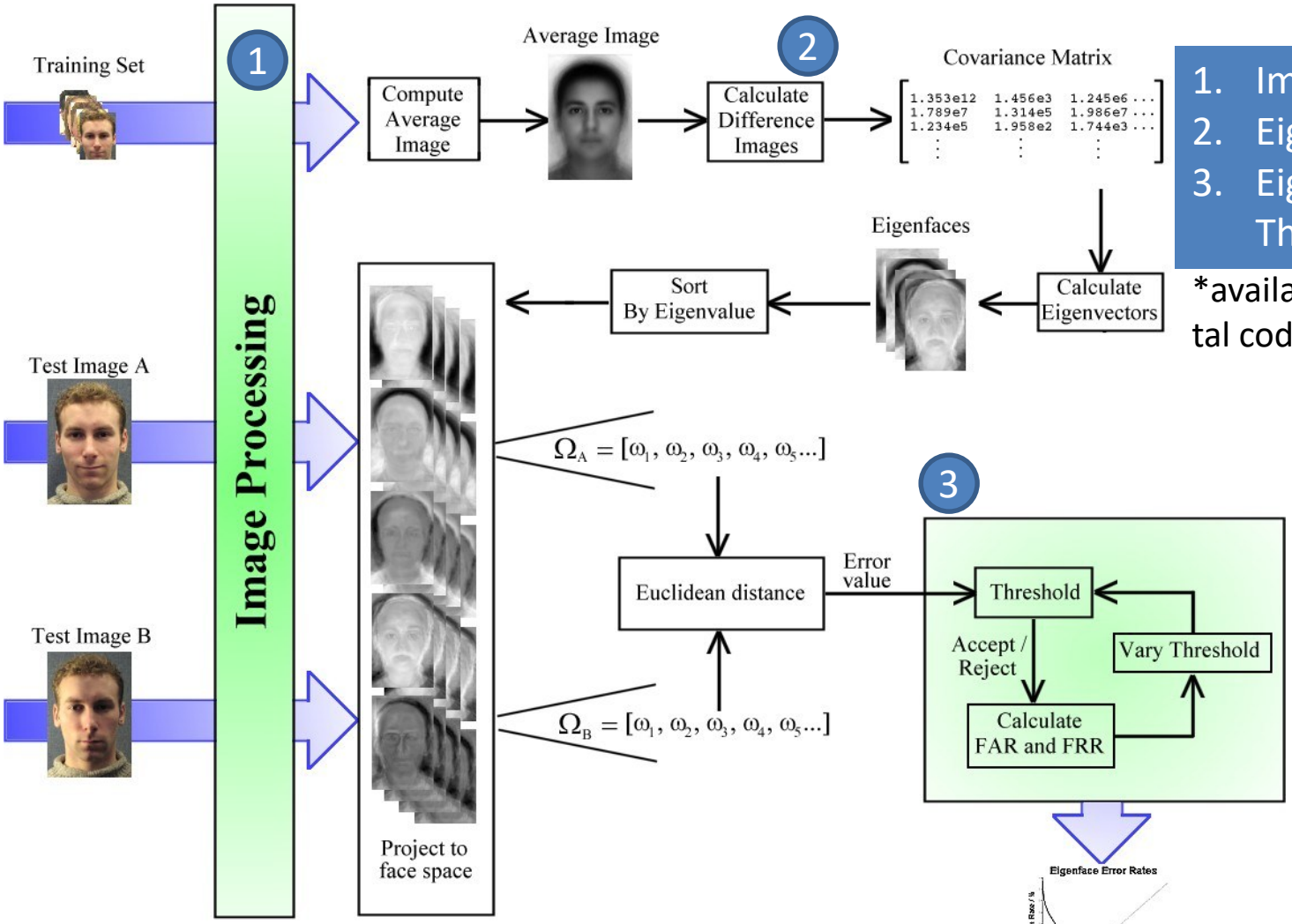
- Input: cropped face images from Yale Face Database B





- Any face can be expressed as linear combination of eigenfaces
- Face can be recognized based on the characteristic coefficients of its eigenfaces linear combination

System



1. Image Pre-processing
2. Eigen Decomposition*
3. Eigenvalue Similarity Threshold*

*available in folder as skeletal code (Matlab)

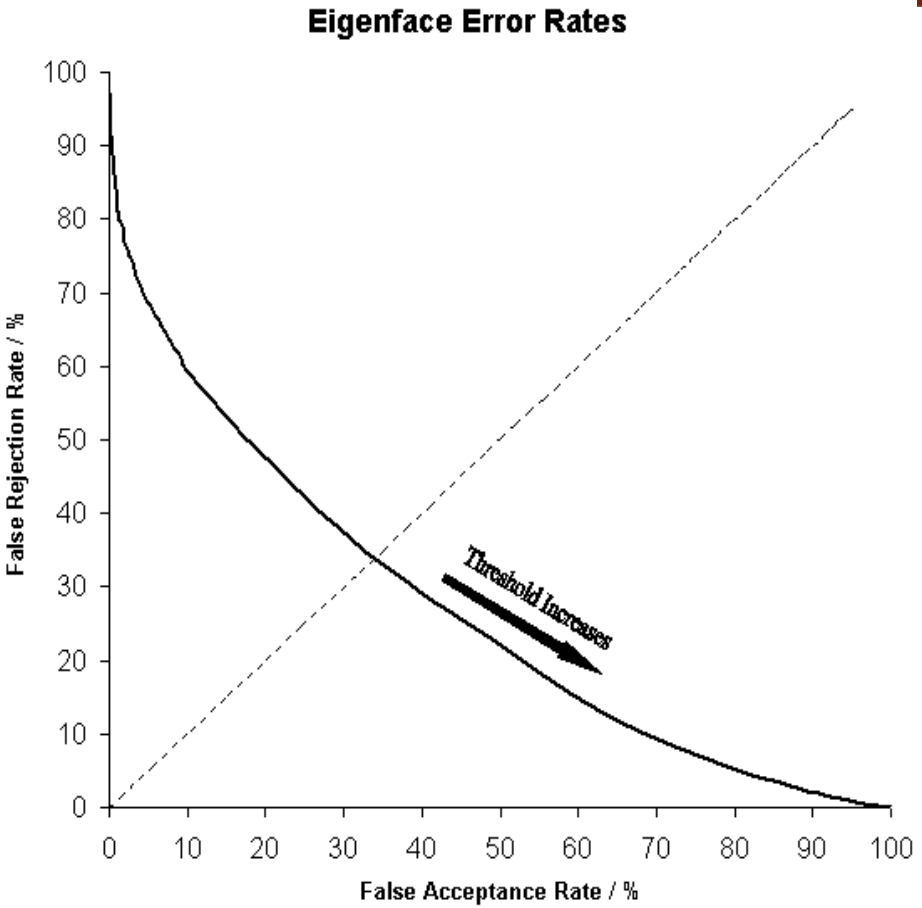
The identity of test entry A is confirmed as the identitiy of database entry-X, if the distance between two entries is lower than a threshold

Output: Equal Error Rates

FAR (False Acceptance Rate)
Security (impostor) risk -- the percentage of below the threshold distance measures, when images of different people are being compared.

FRR (False Rejection Rate)
Convenience (rejection) risk -- the percentage of above the thres-hold distance measures, when images of the same person are being compared.

Lower EER → better recognition



Equal Error Rates (EER): point where FAR = FRR

Problems

Eigenface method suffers from the following limitations:

- **Variations in lighting conditions**
  - Different lighting conditions for enrollment and query cause inaccurate result
- **High computational load with higher image resolution**
  - Higher resolution images may bring greater accuracy, but require more computations
- **Number of eigenface components needs to be optimized for effective comparison test**
  - Comparison test requires more computations when more eigenface components are considered

Yale Face Database B

- Original:

  5760 grayscale face images; 10 subjects, 9 pose variations, 64 lighting conditions

- Subset for this project:
  - Total 310 images; 10 subjects, 1 pose (frontal), 3 types of lighting conditions leading to 3 data subsets:

	Subset 0	**Subset 1**	**Subset 2**
Role	Training set	Test set (compulsory)	Test set (bonus)
Total Image	70	120	120
Total Subject	10	10	10
Image/subject	7	12	12
Lighting condition	Uniform, no shadow in face	Less uniform, more shadow in face	Least uniform, severe shadow

Yale Face Database B

- Original database contains full-face shots, i.e. with hair, ears, and neck. Subset for this project (yaleBfaces) has been cropped such that only the face area (i.e. two eyes, nose, and lips on facial skin background) exists in each image.
  - Image size: 50 x 50 pixels, 8-bit grayscale (png format)
- Data structure
  - Main folder yaleBfaces contains 3 subfolders corresponding to 3 subsets of lighting conditions (subset 0, subset 1, subset 2)
  - Within each subfolder, images are named as: personXX_YY.png, XX denotes the subject identity; YY denotes the lighting conditions.
    - $01 \leq XX \leq 10$ (10 subjects)
    - $01 \leq YY \leq 31$ (31 lighting conditions: 01-07 @subset 0, 08-19 @subset 1, 20-31 @ subset 2)
  - Example: person01_05.png in subset 0 corresponds to the 5th image of subject-01, taken under lighting condition specified in subset 0

Matlab Codes

- To load images & their corresponding label (subject ID) in Matlab, use `loadSubset.m` :

```
[imgs, labels] = loadSubset(subsets, yaleDir)
```

  - yaleDir is the location path of the folder yaleBfaces on your computer; specify it in the Matlab code before running
  - Images are loaded as 1-D vector of 2500 elements (50x50 pixels flattened into 1-D)
  - Label/subject IDs is given as 1-D vector, each element can have values between 1-10
- To build the eigenfaces from a training set and test the recognition of a test set, `run face_recognition.m` :

```
[acc,mindist,recog] = face_recognition(w, labels, w2,
labels2,N)
```

  - Check .m file for further explanations. This is *brute-force eigenface recognition* (i.e. no designated distance threshold, just find the minimum distance)

Matlab Codes

- To explore the eigenface characteristics in batch, build a main program where you call the two previous Matlab functions and add your own batch processing codes.

  Example: finding relation of eigenface number & recognition accuracy (`main_facerec.m`)

- Explore the codes:
  1. Try to extract & display a single image from the output variable `imgs` produced by `loadSubset.m`.
  2. Try to display all images belonging to a subset on a single figure (e.g. a 10 x 7 grids of 50x50 face image from subset 0)
  3. Identify which variables in `face_recognition.m` are the eigenface components and the weights of eigenface linear combinations
  4. Knowing which ones are the eigenface & the weight, reconstruct a face entry using N eigenface and their corresponding weights. See what happens when you use less N.

Task I: Image Preprocessing

Working with the Brute-Force Eigenface Recognition Mode:

1. Find the relation of eigenface number & recognition accuracy based on eigenfaces derived from subset 0, to images in subsets 0, 1 and 2, without any image pre-processing algorithm (determine the number of eigenface first, aim for at least 90% accuracy in subset 0).
2. Propose a solution (pre-processing algorithm) to overcome the problem of illumination sensitivity (*without changing the number of eigenface from #1*)
   a. Repeat #1 after adding image pre-processing algorithm *only to images of subsets 1 and 2*, prior to performing recognition
   b. Explain how the pre-processing algorithm work and compare at least two types of preprocessing algorithm
3. Explore the minimum image size & minimum number of eigenface required to maintain -2% and -5% accuracy from the accuracy you get on #1! (Example: if in #1 your best eigenface accuracy with pre-processing is 85%; then you need to find the minimum image size & minimum number of eigenface components that would give you 83% and 80% accuracy)

Task II: FAR, FFR, EER

Supposed you are working with the Threshold-based Eigenface Recognition Mode:

4. How would you find a good initial guess for a threshold value?
5. How would you modify `face_recognition.m` to implement the threshold-based recognition? In your answer:
   - Please add `th` as an input argument representing threshold value
   - Specify which lines would you modify from the original code so that the output of `face_recognition.m` will represent best match based on threshold
   - Explain how the replacement code works
   - Include a work-around in your code to mitigate situations where there are multiple entries in the dataset that meet a given recognition threshold
6. How would you build a procedure to generate a FAR/FRR/EER?
   - Create a dedicated Matlab function where the output arguments are FAR, FRR, and EER values.
   - You may choose your own input arguments, please provide explanations.